

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

ОТЧЕТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ РАБОТЫ №3
по дисциплине «Методы тестирования программного
обеспечения»
на тему «Статическое тестирование»

Выполнил студент группы
№5130201/30101

Радионова П. В. _____

Проверил преподаватель

Курочкин М. А. _____

« ____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Постановка задачи	4
2 Статический анализ кода	5
3 Описание тестируемой программы	6
4 Описание приложения и среды разработки	16
5 Запуск CppCheck	18
6 Результаты запуска	19
7 Внесение исправлений	20
Заключение	29

Введение

Целесообразность вычитывания кода зависит от ряда факторов: масштаб и сложность приложения, численный состав команды разработчиков, временной график проекта (например, щадящий он или напряженный) и, конечно же, квалификация и профессиональная культура команды программистов.

Статический анализ кода — анализ исходного кода программного обеспечения, производимый без реального выполнения исследуемых программ.

Сррсheck — статический анализатор кода для языков С и С++, предназначенный для поиска проблем, которые не обнаруживаются компиляторами.

1 Постановка задачи

Цель работы: провести статический анализ программы.

Задачи.

1. Изучить методы статического анализа.
2. Провести статический анализ программы при помощи CppCheck.
3. Внести исправления в код на основе результатов статического анализа.

2 Статический анализ кода

Статический анализ кода – это метод исследования исходного кода программы без её выполнения. Путем анализа исходного кода происходит выявление потенциальных ошибок, уязвимостей и нарушения стандартов кодирования. Статический анализ также может использоваться для проверки стиля кода, что является полезным для поддержания единообразия стиля в больших проектах. Инструменты статического анализа могут быть интегрированы в процесс сборки, что позволяет автоматизировать проверку кода.

Виды статического тестирования

1. Статический анализ кода. Статический анализ кода представляет собой анализ исходного кода. Целью такого анализа является выявление потенциальных проблем, структурных аномалий и нарушений стандартов кодирования.
2. Обзоры кода. Вид статического тестирования, включающий в себя анализ кода членами команды разработки и экспертами.
3. Анализ архитектуры. Вид тестирования, при котором анализируется архитектура ПО: его структура, зависимости между компонентами, соответствие архитектурным принципам.

3 Описание тестируемой программы

Название программы

Вычисление кратчайшего пути в связном ациклическом неориентированном графе методом Беллмана-Форда.

Входные данные

- `int vertex` – количество вершин в графе;
- `int in` – номер действия пользовательского интерфейса;
- `int start, finish` – индексы начальной и конечной вершины пути на графе.

Ограничения

Все параметры – целые числа:

- $vertex \in [2, 100]$;
- $in \in [0, 2]$;
- $start \in [1, vertex]$;
- $finish \in [1, vertex]$.

Спецификация

В таблице 1 приведена спецификация программы: выходные данные и реакция программы на граничные условия входных данных.

Таблица 1: Таблица входных и выходных данных программы

Входные данные				Выходные дан- ные	Реакция про- граммы
vertex	in	start	finish		
$\notin [2, 100]$	-	-	-	Сообщение: «Please, enter the integer from 2 to 100»	Повторный запрос ввода до тех пор, пока не будет вве- дено допустимое значение коли- чества вершин в графе.
$\in [2, 100]$	-	-	-	Степени вершин графа, матрица весов графа и меню действий.	Ожидание ввода пользователя.
$\in [2, 100]$	$\notin [0, 2]$	-	-	Сообщение: «Please, enter the integer number from 0 to 2»	Повторный запрос ввода до тех пор, пока не будет вве- дено допустимое значение действия меню.
$\in [2, 100]$	2	-	-	Сообщение: «Enter the number of vertex you want to start from (from 1 to vertex):»	Программа ожи- дает ввода номера стартовой верши- ны пути.
$\in [2, 100]$	2	$\notin [1, \text{vertex}]$	-	Сообщение: «Input error. Please, enter the value from 0 to vertex»	Повторный запрос ввода до тех пор, пока не будет вве- дено допустимое значение.
$\in [2, 100]$	2	$\in [1, \text{vertex}]$	-	Сообщение: «Enter the number of vertex you want to finish (from 1 to vertex):»	Программа ожи- дает ввода номера конечной верши- ны пути.
$\in [2, 100]$	2	$\in [1, \text{vertex}]$	$\notin [1, \text{vertex}]$	Сообщение: «Input error. Please, enter the value from 0 to vertex»	Повторный запрос ввода до тех пор, пока не будет вве- дено допустимое значение.
$\in [2, 100]$	2	$\in [1, \text{vertex}]$	$\in [1, \text{vertex}]$	Сообщение с дли- ной минимального пути, меню дей- ствий	Ожидание ввода пользователя.
$\in [2, 100]$	1	-	-	Сообщение: «Enter the number of vertices: »	Программа ожи- дает ввод количе- ства вершин для нового графа
$\in [2, 100]$	0	-	-	-	Завершение рабо- ты.

Код программы

```
1  #pragma once
2  #include <iostream>
3  #include <vector>
4  #include <queue>
5  #include <map>
6  #include <set>
7  #include <unordered_set>
8  #include <stack>
9  #include <random>
10 #include <algorithm>
11 #include <numeric>
12 #include <limits>
13 #include <ctime>
14 #include <iomanip>
15 #include <string>
16 #include <fstream>
17 #include <sstream>
18
19 class Graph
20 {
21 private:
22     int numOfVertex;
23     std::vector<int> degrees;
24     std::vector<std::vector<int>> adjacencyMatrix;
25     std::vector<std::vector<int>> weightMatrix;
26
27     double randomDouble() {
28         static std::random_device rd;
29         static std::mt19937 gen(rd());
30         static std::uniform_real_distribution<> dis(0.0, 1.0);
31         return dis(gen);
32     }
33 public:
34     Graph() : numOfVertex(0) {}
35     Graph(int vertex);
36
37     void generateDegrees(double lambda);
38     void createAdjMatrix();
39     void generateWeightMatrix(double lambda);
40
41     bool BellmanFord(int start, int finish, std::vector<int>&
42         distances, std::vector<int>& path, int& iterations);
43
44     int getNumOfVertex() const;
45     const std::vector<int> getDegrees();
46     const std::vector<std::vector<int>> getAdjMatrix() const;
47     const std::vector<std::vector<int>> getWeightMatrix() const;
48 };
```

Листинг 1: Исходный код Graph.h

```

1  #include "Graph.h"
2
3  Graph::Graph(int vertex) : numOfVertex(vertex)
4  {
5      adjacencyMatrix.resize(numOfVertex, std::vector<int>(
6          numOfVertex, 0));
7      weightMatrix.resize(numOfVertex, std::vector<int>(numOfVertex
8          , 0));
9      degrees.resize(numOfVertex, 0);
10 }
11
12 int Graph::getNumOfVertex() const { return numOfVertex; }
13
14 const std::vector<int> Graph::getDegrees() { return degrees; }
15
16 const std::vector<std::vector<int>> Graph::getAdjMatrix() const {
17     return adjacencyMatrix; }
18
19 const std::vector<std::vector<int>> Graph::getWeightMatrix()
20     const { return weightMatrix; }
21
22 void Graph::generateDegrees(double lambda) {
23     degrees.clear();
24     degrees.resize(numOfVertex, 0);
25
26     for (int i = 0; i < numOfVertex; ++i) {
27         double r_i = randomDouble();
28         r_i = std::max(r_i, 1e-10);
29
30         double x_i = -log(r_i) / lambda;
31
32         int degree = static_cast<int>(round(x_i));
33
34         int max_possible_degree = numOfVertex - i - 1;
35         if (max_possible_degree > 0) {
36             degree = std::min(degree, max_possible_degree);
37         }
38         else {
39             degree = 0;
40         }
41
42         degrees[i] = std::max(degree, 0);
43     }
44
45     std::sort(degrees.begin(), degrees.end(), std::greater<int>())
46         );
47
48     bool isValid = false;
49     while (!isValid) {
50         isValid = true;

```

```

47     for (int i = 0; i < numOfVertex; ++i) {
48         if (degrees[i] > numOfVertex - i - 1) {
49             isValid = false;
50             break;
51         }
52     }
53
54     if (!isValid) {
55         for (int i = 0; i < numOfVertex; ++i) {
56             double r_i = randomDouble();
57             r_i = std::max(r_i, 1e-10);
58             double x_i = -log(r_i) / lambda;
59             int degree = static_cast<int>(round(x_i));
60             int max_possible_degree = numOfVertex - i - 1;
61             degrees[i] = std::min(std::max(degree, 0),
62                                   max_possible_degree);
63         }
64         std::sort(degrees.begin(), degrees.end(), std::greater<int>());
65     }
66
67     if (numOfVertex == 2) {
68         degrees[0] = 1;
69         degrees[1] = 0;
70     }
71     degrees[numOfVertex - 1] = 0;
72 }
73
74 void Graph::createAdjMatrix()
75 {
76     adjacencyMatrix.assign(numOfVertex, std::vector<int>(
77         numOfVertex, 0));
78     std::vector<int> remaining_degrees(degrees);
79
80     for (int i = 0; i < numOfVertex; ++i) {
81         while (remaining_degrees[i] > 0) {
82             int max_possible_j = numOfVertex - 1;
83             int available = max_possible_j - i;
84
85             if (available <= 0) {
86                 remaining_degrees[i] = 0;
87                 break;
88             }
89
90             int j = i + 1 + (rand() % available);
91
92             if (adjacencyMatrix[i][j] == 0) {
93                 adjacencyMatrix[i][j] = 1;
94                 remaining_degrees[i]--;
95             }
96         }
97     }

```

```

96     }
97 };
98
99 void Graph::generateWeightMatrix(double lambda)
100 {
101     static std::random_device rd;
102     static std::mt19937 gen(rd());
103
104     for (int i = 0; i < numOfVertex; i++) {
105         for (int j = 0; j < numOfVertex; j++) {
106             if (adjacencyMatrix[i][j] != 0) {
107                 double U = std::uniform_real_distribution<>(0.0,
108                     1.0)(gen);
109
110                 double random_value = -std::log(1 - U) / lambda;
111
112                 weightMatrix[i][j] = static_cast<int>(
113                     random_value) + 1;
114             }
115         }
116     }
117
118 bool Graph::BellmanFord(int start, int finish, std::vector<int>&
119     distances, std::vector<int>& path, int& iterations)
120 {
121     distances.assign(numOfVertex, INT_MAX);
122     path.assign(numOfVertex, -1);
123     distances[start] = 0;
124     iterations = 0;
125
126     for (int i = 0; i < numOfVertex - 1; ++i) {
127         bool updated = false;
128         for (int u = 0; u < numOfVertex; ++u) {
129             for (int v = 0; v < numOfVertex; ++v) {
130                 if (adjacencyMatrix[u][v] && weightMatrix[u][v]
131                     != 0) { // Если есть ребро u->v
132                     iterations++;
133                     if (distances[u] != INT_MAX && distances[u] +
134                         weightMatrix[u][v] < distances[v]) {
135                         distances[v] = distances[u] +
136                             weightMatrix[u][v];
137                         path[v] = u;
138                         updated = true;
139                     }
140                 }
141             }
142         }
143         if (!updated) break;
144     }
145
146     for (int u = 0; u < numOfVertex; ++u) {

```

```

142         for (int v = 0; v < numOfVertex; ++v) {
143             if (adjacencyMatrix[u][v] && weightMatrix[u][v] != 0)
144                 {
145                     if (distances[u] != INT_MAX && distances[u] +
146                         weightMatrix[u][v] < distances[v]) {
147                         std::cerr << "Graph contains negative-weight
148                             cycle!" << std::endl;
149                         return false;
150                     }
151                 }
152             }
153         }
154     }
155     if (distances[finish] == INT_MAX) {
156         return false;
157     }
158     return true;
159 }

```

Листинг 2: Исходный код Graph.cpp

```

1  #include "Graph.h"
2
3  const double lambda = 0.2;
4  const double lambda2 = 0.25;
5
6  void printMatrix(std::vector<std::vector<int>> matrix, std::
7      string label)
8  {
9      std::cout << label << ":\n";
10     for (int i = 0; i < matrix.size(); i++) {
11         for (int j = 0; j < matrix.size(); j++) {
12             std::cout << std::setw(3) << matrix[i][j] << std::
13                 setw(3);
14         }
15         std::cout << "\n";
16     }
17     std::cout << "\n";
18 }
19
20 int input()
21 {
22     int value;
23     while (true) {
24         std::cin >> value;
25         if (std::cin.peek() != '\n') {
26             std::cin.clear();
27             std::cin.ignore(std::cin.rdbuf()->in_avail());
28             printf("Input error. Please, enter the integer value
29                 .\n");
30         }
31         else break;
32     }
33 }

```

```

29     }
30     return value;
31 }
32
33 void getStartAndFinish(int& start, int& finish, int numOfVertex)
34 {
35     std::cout << "Enter the number of vertex you want to start
        from (from 1 to " << numOfVertex << "): \n";
36     start = input();
37     while (start<1 || start>numOfVertex)
38     {
39         std::cout << "Input error. Please, enter the value from 0
        to " << numOfVertex << ".\n";
40         start = input();
41     }
42     std::cout << "Enter the number of vertex you want to finish (
        from 1 to " << numOfVertex << "): \n";
43     finish = input();
44     while (finish<1 || finish>numOfVertex)
45     {
46         std::cout << "Input error. Please, enter the value from 1
        to " << numOfVertex << ".\n";
47         finish = input();
48     }
49 }
50
51 void printMenu()
52 {
53     printf("\nEnter the number of option:\n");
54     printf("[0] - Exit\n");
55     printf("[1] - Create a new graph\n");
56     printf("[2] - Bellman - Ford's algorithm\n");
57     printf("\nYour choice: ");
58 }
59
60 int main()
61 {
62     bool flag = true;
63     while (flag) {
64         int vertex;
65         std::cout << "Enter the number of vertices: ";
66         vertex = input();
67         while (vertex < 2 || vertex>100) {
68             std::cout << "Please, enter the integer from 2 to
        100.\n";
69             vertex = input();
70         }
71
72         int edges = vertex - 1;
73         Graph graph(vertex);
74         graph.generateDegrees(lambda);
75

```

```

76     for (int i = 0; i < graph.getNumOfVertex(); i++) {
77         std::cout << "Degree of the vertex num." << i + 1 <<
            ": " << graph.getDegrees()[i] << "\n";
78     }
79     graph.createAdjMatrix();
80
81     graph.generateWeightMatrix(lambda2);
82
83     printMatrix(graph.getWeightMatrix(), "\nWeight matrix");
84
85     printMenu();
86     int in;
87     bool anotherFlag = true;
88     while (anotherFlag) {
89         in = input();
90         switch (in) {
91             case 0:
92             {
93                 flag = false;
94                 anotherFlag = false;
95                 break;
96             }
97             case 1:
98             {
99                 std::cout << "You chose [1] - Create a new graph\
100                     n";
101                 anotherFlag = false;
102                 break;
103             }
104             case 2:
105             {
106                 std::cout << "You chose [2] - Bellman - Ford's
107                     algorithm\n";
108                 int start, finish, iterations;
109                 std::vector<int> distances, path;
110                 getStartAndFinish(start, finish, graph.
111                     getNumOfVertex());
112                 start--; finish--;
113                 if (graph.BellmanFord(start, finish, distances,
114                     path, iterations)) {
115                     std::cout << "Shortest distance from " <<
116                         start + 1 << " to " << finish + 1 << ": "
117                         << distances[finish] << "\n";
118
119                     std::vector<std::pair<int, int>> edges;
120                     for (int v = finish; v != start; v = path[v])
121                     {
122                         if (path[v] == -1) break;
123                         edges.push_back(std::make_pair(path[v], v
124                             ));
125                     }
126                     std::reverse(edges.begin(), edges.end());

```

```

119         std::cout << "Path edges:\n";
120         for (const auto& edge : edges) {
121             int u = edge.first;
122             int v = edge.second;
123             std::cout << "( " << u + 1 << " -> " << v
124                 + 1 << " ) weight: " << graph.
125                 getWeightMatrix()[u][v] << "\n";
126         }
127     }
128     else {
129         std::cout << "No path exists between " <<
130             start + 1 << " and " << finish + 1 << ".\n"
131             ";
132     }
133     printMenu();
134     break;
135 }
136 default:
137 {
138     std::cout << "Please, enter the integer number
139         from 0 to 3.\n";
140     printMenu();
141     break;
142 }
143 }
144 }
145 }

```

Листинг 3: Исходный код main.cpp

4 Описание приложения и среды разработки

CppCheck – программное обеспечение для статического анализа кода на языках C/C++. Он позволяет выявлять проблемы, уязвимости и возможности для оптимизации без запуска программы. Тестирование может быть проведено как через командную строку, так и с помощью графического интерфейса.

CppCheck анализирует код на наличие проблем и классифицирует их по степени серьезности:

- **Error:** ошибки, которые могут привести к сбою программы.
- **Warning:** предупреждения – проблемы, которые могут вызвать некорректное поведение программы.
- **Style:** стилистические проблемы, которые не влияют на выполнимость программы, но ухудшают читаемость кода.
- **Performance:** рекомендации по улучшению производительности кода.
- **Portability:** проблемы, связанные с переносимостью кода на разные платформы.
- **Information:** информационные сообщения.

CppCheck поддерживает анализ данных в двух форматах: текстовом и в xml формате.

Правила проверки CppCheck проверяет код на соответствие **167** правилам, которые разделены на следующие категории:

- | | |
|------------------------|-------------------------------------|
| • 64-bit portability | • Condition |
| • Assert | • Exception Safety |
| • Auto Variables | • IO using format stringstyle |
| • Boolean | • Leaks (auto variables) |
| • Boost usage | • Memory leaks (address not taken) |
| • Bounds checking | • Memory leaks (class variables) |
| • Check function usage | • Memory leaks (function variables) |
| • Class | • Memory leaks (struct members) |

- Null pointer
- Other
- STL usage
- Sizeof
- String
- Type
- Uninitialized variables
- Unused functions
- UnusedVar
- Using postfix operators
- Vaarg

Разбор правил из группы «64-bit portability» Группа включает 4 ключевых сценария, которые CppCheck может обнаружить.

1. Присваивание адреса целому числу (AssignmentAddressToInteger).
 - Ищет присваивание значения указателя (T^*) переменной целого типа (int, long).
 - Проблема в том, что адрес (64 бита) не помещается в int (32 бита). В результате старшая часть адреса отбрасывается, и указатель становится некорректным.
2. Присваивание целого числа указателю (AssignmentIntegerToAddress).
 - Ищет присваивание целого числа (например, константы) переменной указателю.
3. Возврат адреса из функции с целочисленным типом (CastAddressToIntegerAtReturn).
 - Ищет функции, которые по объявлению возвращают int или long, но в теле возвращают указатель.
4. Возврат целого числа из функции с типом-указателем (CastIntegerToAddressAtReturn).
 - Ищет функции, которые по объявлению возвращают указатель, а в теле возвращают целое число.

Эта группа помогает сделать код переносимым. Игнорирование таких предупреждений на 32-битной системе может быть незаметно, но при компиляции для 64-битной платформы мы получим логические ошибки или повреждение данных

5 Запуск CppCheck

Запуск CppCheck для выполнения лабораторной работы осуществляется через командную строку.

Порядок запуска

1. Скачать и распаковать CppCheck.
2. Открыть командную строку (нажать Win + R, ввести cmd).
3. Перейти в директорию с `cppcheck.exe`:

```
cd путь_к_директории.
```

4. Выполнить команду:

```
cppcheck -enable=all -inconclusive путь_к_исходным_файлам.
```

5. Для просмотра разных видов ошибок использовать следующие параметры:

- `-enable=style` для проверки стиля кода,
- `-enable=performance` для проверки производительности,
- `-enable=portability` для проверки переносимости кода,
- `-enable=information` для получения информационных сообщений,
- `-enable=unusedFunction` для поиска неиспользуемых функций,
- `-enable=warning` для отображения предупреждений.

6 Результаты запуска

По результатам запуска была произведена проверка по 167 правилам. После выполнения команды

```
cppcheck -enable=all -inconclusive Graph.h Graph.cpp main.cpp
```

было выявлено следующее:

- 2 предупреждения о производительности (performance),
- 5 стилистических предупреждений (style).

Описание найденных проблем

1. Проблемы производительности.

- (a) Function parameter 'matrix' should be passed by const reference.
- (b) Function parameter 'label' should be passed by const reference.

Описание: параметры `matrix` и `label` в функции `printMatrix` передаются по значению, что вызывает копирование всего вектора и строки.

2. Стилистические ошибки.

- (a) The member function 'Graph::randomDouble' can be static.
- (b) Technically the member function 'Graph::getDegrees' can be const.
- (c) The function 'getAdjMatrix' is never used.
- (d) Class 'Graph' has a constructor with 1 argument that is not explicit.
- (e) The scope of the variable 'in' can be reduced.

Описание: ошибка (a) указывает на то, что метод `randomDouble` не использует нестатические члены класса, но не объявлен статическим; ошибка (b) указывает на то, что метод `getDegrees` не изменяет состояние объекта, но не указан как `const`; ошибка (c) указывает на неиспользуемую функцию; ошибка (d) указывает на конструктор класса `Graph` с одним параметром, который не объявлен как `explicit`, что может привести к неявным преобразованиям типов; (e) указывает на то, что переменная `in` объявлена во внешнем блоке, но используется только внутри цикла.

7 Внесение исправлений

Изменения

1. Передача параметров по константной ссылке: в функции printMatrix параметры matrix и label теперь передаются по константной ссылке, что улучшает производительность.
2. Функция randomDouble теперь объявлена как статическая.
3. Функция getDegrees теперь объявлен как const.
4. Конструктор класса Graph с одним параметром теперь объявлен как explicit, чтобы избежать неявных преобразований типов.
5. Переменная in теперь объявлена внутри цикла.
6. Лишний метод getAdjMatrix был удален.

После внесения всех исправлений в код программы CppCheck сообщил об отсутствии проблем.

Исправленный код

```
1  #include <iostream>
2  #include <vector>
3  #include <random>
4  #include <algorithm>
5  #include <limits>
6  #include <iomanip>
7
8  class Graph
9  {
10 private:
11     static constexpr int MAX_VERTEX_DEGREE = 100;
12     static constexpr double MIN_RANDOM_VALUE = 1e-10;
13     static constexpr int MAX_GENERATION_ATTEMPTS = 1000;
14     static constexpr int INFINITY_DISTANCE = INT_MAX;
15
16     int numOfVertex;
17     std::vector<int> degrees;
18     std::vector<std::vector<int>> adjacencyMatrix;
19     std::vector<std::vector<int>> weightMatrix;
20
21     static double randomDouble() {
22         static std::random_device rd;
23         static std::mt19937 gen(rd());
24         static std::uniform_real_distribution<> dis(0.0, 1.0);
25         return dis(gen);
```

```

26     }
27
28     bool isGraphicSequence(std::vector<int> degs);
29
30 public:
31     Graph() : numOfVertex(0) {}
32     explicit Graph(int vertex);
33
34     void generateDegrees(double lambda);
35     void createAdjMatrix();
36     void generateWeightMatrix(double lambda);
37
38     bool BellmanFordAlgorithm(int start, int finish, std::vector<
        int>& distances, std::vector<int>& path, int& iterations);
39
40     int getNumOfVertex() const;
41     const std::vector<int>& getDegrees() const;
42     const std::vector<std::vector<int>>& getWeightMatrix() const;
43 };

```

Листинг 4: Исправленный код Graph.h

```

1  #include "Graph.h"
2  #include <algorithm>
3  #include <functional>
4  #include <numeric>
5  #include <cmath>
6
7  Graph::Graph(int vertex) : numOfVertex(vertex)
8  {
9      adjacencyMatrix.resize(numOfVertex, std::vector<int>(
        numOfVertex, 0));
10     weightMatrix.resize(numOfVertex, std::vector<int>(numOfVertex
        , 0));
11     degrees.resize(numOfVertex, 0);
12 }
13
14 int Graph::getNumOfVertex() const { return numOfVertex; }
15
16 const std::vector<int>& Graph::getDegrees() const { return
    degrees; }
17
18 const std::vector<std::vector<int>>& Graph::getWeightMatrix()
    const { return weightMatrix; }
19
20 bool Graph::isGraphicSequence(std::vector<int> degs)
21 {
22     while (true) {
23         std::sort(degs.begin(), degs.end(), std::greater<int>());
24
25         while (!degs.empty() && degs.back() == 0) {
26             degs.pop_back();

```

```

27     }
28
29     if (degs.empty()) {
30         return true;
31     }
32
33     int d = degs.front();
34     degs.erase(degs.begin());
35
36     if (d > degs.size()) {
37         return false;
38     }
39
40     for (int i = 0; i < d; ++i) {
41         degs[i]--;
42         if (degs[i] < 0) {
43             return false;
44         }
45     }
46 }
47 }
48
49 void Graph::generateDegrees(double lambda)
50 {
51     degrees.clear();
52     degrees.resize(numOfVertex, 0);
53
54     for (int attempt = 0; attempt < MAX_GENERATION_ATTEMPTS; ++
55         attempt) {
56         for (int i = 0; i < numOfVertex; ++i) {
57             double r_i = randomDouble();
58             r_i = std::max(r_i, MIN_RANDOM_VALUE);
59             double x_i = -log(r_i) / lambda;
60             degrees[i] = std::min(static_cast<int>(round(x_i)),
61                 MAX_VERTEX_DEGREE);
62         }
63
64         std::sort(degrees.begin(), degrees.end(), std::greater<
65             int>());
66
67         int sum = std::accumulate(degrees.begin(), degrees.end(),
68             0);
69         if (sum % 2 != 0) {
70             continue;
71         }
72
73         if (isGraphicSequence(degrees)) {
74             std::cout << "Graphic sequence generated after " <<
75                 attempt + 1 << " attempts.\n";
76             return;
77         }
78     }
79 }

```

```

74
75     std::cout << "Using fallback: generating a path graph.\n";
76     degrees.assign(numOfVertex, 2);
77     degrees[0] = 1;
78     degrees[numOfVertex - 1] = 1;
79
80     if (numOfVertex == 2) {
81         degrees[0] = 1;
82         degrees[1] = 1;
83     }
84 }
85
86 void Graph::createAdjMatrix()
87 {
88     adjacencyMatrix.assign(numOfVertex, std::vector<int>(
89         numOfVertex, 0));
90
91     std::vector<std::pair<int, int>> vertices;
92     for (int i = 0; i < numOfVertex; ++i) {
93         vertices.push_back({ degrees[i], i });
94     }
95
96     while (true) {
97         std::sort(vertices.begin(), vertices.end(),
98             [](const std::pair<int, int>& a, const std::pair<int,
99                 int>& b) {
100                 return a.first > b.first;
101             });
102
103         while (!vertices.empty() && vertices.back().first == 0) {
104             vertices.pop_back();
105         }
106
107         if (vertices.empty()) {
108             break;
109         }
110
111         int d = vertices[0].first;
112         int u_index = vertices[0].second;
113
114         vertices.erase(vertices.begin());
115
116         if (d > vertices.size()) {
117             std::cerr << "Error in Havel-Hakimi algorithm:
118                 insufficient vertices.\n";
119             break;
120         }
121
122         for (int i = 0; i < d; ++i) {
123             vertices[i].first--;
124             int v_index = vertices[i].second;
125             adjacencyMatrix[u_index][v_index] = 1;

```

```

123     }
124 }
125
126 for (int i = 0; i < numOfVertex; ++i) {
127     for (int j = i + 1; j < numOfVertex; ++j) {
128         if (adjacencyMatrix[i][j] == 1) {
129             adjacencyMatrix[j][i] = 1;
130         }
131     }
132 }
133 }
134
135 void Graph::generateWeightMatrix(double lambda)
136 {
137     static std::random_device rd;
138     static std::mt19937 gen(rd());
139
140     for (int i = 0; i < numOfVertex; i++) {
141         for (int j = 0; j < numOfVertex; j++) {
142             if (adjacencyMatrix[i][j] != 0) {
143                 double U = std::uniform_real_distribution<>(0.0,
144                     1.0)(gen);
145                 double random_value = -std::log(1 - U) / lambda;
146                 weightMatrix[i][j] = static_cast<int>(
147                     random_value) + 1; // +1 чтобы вес был миниму
148                                     м 1
149             }
150         }
151     }
152 }
153
154 bool Graph::BellmanFordAlgorithm(int start, int finish, std::
155     vector<int>& distances, std::vector<int>& path, int&
156     iterations)
157 {
158     distances.assign(numOfVertex, INFINITY_DISTANCE);
159     path.assign(numOfVertex, -1);
160     distances[start] = 0;
161     iterations = 0;
162
163     for (int i = 0; i < numOfVertex - 1; ++i) {
164         bool updated = false;
165         for (int u = 0; u < numOfVertex; ++u) {
166             for (int v = 0; v < numOfVertex; ++v) {
167                 if (adjacencyMatrix[u][v] && weightMatrix[u][v]
168                     != 0) {
169                     iterations++;
170                     if (distances[u] != INFINITY_DISTANCE &&
171                         distances[u] + weightMatrix[u][v] <
172                             distances[v]) {
173                         distances[v] = distances[u] +
174                             weightMatrix[u][v];

```

```

167         path[v] = u;
168         updated = true;
169     }
170 }
171 }
172 }
173 if (!updated) break;
174 }
175
176 for (int u = 0; u < numOfVertex; ++u) {
177     for (int v = 0; v < numOfVertex; ++v) {
178         if (adjacencyMatrix[u][v] && weightMatrix[u][v] != 0)
179         {
180             if (distances[u] != INFINITY_DISTANCE &&
181                 distances[u] + weightMatrix[u][v] < distances
182                     [v]) {
183                 std::cerr << "Graph contains negative-weight
184                     cycle!" << std::endl;
185                 return false;
186             }
187         }
188     }
189 }
190
191 return distances[finish] != INFINITY_DISTANCE;
192 }

```

Листинг 5: Исправленный код Graph.cpp

```

1  #define _CRT_SECURE_NO_WARNINGS
2  #include "Graph.h"
3
4  const double LAMBDA_DEGREES = 0.2;
5  const double LAMBDA_WEIGHTS = 0.25;
6  const unsigned int RANDOM_SEED = 42;
7  const int MIN_VERTICES = 2;
8  const int MAX_VERTICES = 100;
9  const int MENU_EXIT = 0;
10 const int MENU_NEW_GRAPH = 1;
11 const int MENU_BELLMAN_FORD = 2;
12
13 void printMatrix(const std::vector<std::vector<int>>& matrix,
14                 const std::string& label)
15 {
16     std::cout << label << ":\n";
17     for (int i = 0; i < matrix.size(); i++) {
18         for (int j = 0; j < matrix.size(); j++) {
19             std::cout << std::setw(3) << matrix[i][j] << std::
20                 setw(3);
21         }
22         std::cout << "\n";
23     }
24 }

```

```

22     std::cout << "\n";
23 }
24
25 int validateInput()
26 {
27     int value;
28     while (true) {
29         std::cin >> value;
30         if (std::cin.peek() != '\n') {
31             std::cin.clear();
32             std::cin.ignore(std::cin.rdbuf()->in_avail());
33             printf("Input error. Please, enter the integer value
34                 .\n");
35         }
36         else break;
37     }
38     return value;
39 }
40 void getStartAndFinish(int& start, int& finish, int numOfVertex)
41 {
42     std::cout << "Enter the number of vertex you want to start
43         from (from 1 to " << numOfVertex << "): \n";
44     start = validateInput();
45     while (start < 1 || start > numOfVertex)
46     {
47         std::cout << "Input error. Please, enter the value from 1
48             to " << numOfVertex << ".\n";
49         start = validateInput();
50     }
51     std::cout << "Enter the number of vertex you want to finish (
52         from 1 to " << numOfVertex << "): \n";
53     finish = validateInput();
54     while (finish < 1 || finish > numOfVertex)
55     {
56         std::cout << "Input error. Please, enter the value from 1
57             to " << numOfVertex << ".\n";
58         finish = validateInput();
59     }
60 }
61 void printMenu()
62 {
63     printf("\nEnter the number of option:\n");
64     printf("[%d] - Exit\n", MENU_EXIT);
65     printf("[%d] - Create a new graph\n", MENU_NEW_GRAPH);
66     printf("[%d] - Bellman - Ford's algorithm\n",
67         MENU_BELLMAN_FORD);
68     printf("\nYour choice: ");
69 }
70
71 int main()
72 {

```

```

68     bool flag = true;
69     while (flag) {
70         int vertex;
71         std::cout << "Enter the number of vertices: ";
72         vertex = validateInput();
73         while (vertex < MIN_VERTICES || vertex > MAX_VERTICES) {
74             std::cout << "Please, enter the integer from " <<
75                 MIN_VERTICES << " to " << MAX_VERTICES << ".\n";
76             vertex = validateInput();
77         }
78         Graph graph(vertex);
79         graph.generateDegrees(LAMBDA_DEGREES);
80
81         for (int i = 0; i < graph.getNumOfVertex(); i++) {
82             std::cout << "Degree of the vertex num." << i + 1 <<
83                 ": " << graph.getDegrees()[i] << "\n";
84         }
85         graph.createAdjMatrix();
86         graph.generateWeightMatrix(LAMBDA_WEIGHTS);
87
88         printMatrix(graph.getWeightMatrix(), "\nWeight matrix");
89
90         printMenu();
91
92         bool anotherFlag = true;
93         while (anotherFlag) {
94             int in;
95             in = validateInput();
96             switch (in) {
97                 case MENU_EXIT:
98                     {
99                         flag = false;
100                         anotherFlag = false;
101                         break;
102                     }
103                 case MENU_NEW_GRAPH:
104                     {
105                         std::cout << "You chose [" << MENU_NEW_GRAPH << "
106                             ] - Create a new graph\n";
107                         anotherFlag = false;
108                         break;
109                     }
110                 case MENU_BELLMAN_FORD:
111                     {
112                         std::cout << "You chose [" << MENU_BELLMAN_FORD
113                             << "]" - Bellman - Ford's algorithm\n";
114                         int start, finish, iterations;
115                         std::vector<int> distances, path;
116                         getStartAndFinish(start, finish, graph.
117                             getNumOfVertex());

```

```

115         start--;
116         finish--;
117
118         if (graph.BellmanFordAlgorithm(start, finish,
119             distances, path, iterations)) {
120             std::cout << "Shortest distance from " <<
121                 start + 1 << " to " << finish + 1 << ": "
122                 << distances[finish] << "\n";
123             std::cout << "Number of iterations: " <<
124                 iterations << "\n";
125
126             std::vector<std::pair<int, int>> edges;
127             for (int v = finish; v != start; v = path[v])
128             {
129                 if (path[v] == -1) break;
130                 edges.push_back(std::make_pair(path[v], v));
131             }
132             std::reverse(edges.begin(), edges.end());
133
134             std::cout << "Path edges:\n";
135             for (const auto& edge : edges) {
136                 int u = edge.first;
137                 int v = edge.second;
138                 std::cout << "( " << u + 1 << " -> " << v
139                     + 1 << " ) weight: " << graph.
140                     getWeightMatrix()[u][v] << "\n";
141             }
142         }
143         else {
144             std::cout << "No path exists between " <<
145                 start + 1 << " and " << finish + 1 << ".\n";
146         }
147         printMenu();
148         break;
149     }
150     default:
151     {
152         std::cout << "Please, enter the integer number
153             from " << MENU_EXIT << " to " <<
154             MENU_BELLMAN_FORD << ".\n";
155         printMenu();
156         break;
157     }
158 }
159 }
160 }
161 return 0;
162 }

```

Листинг 6: Исправленный код main.cpp

Заключение

В ходе выполнения лабораторной работы был проведен статический анализ кода при помощи CppCheck. Использование CppCheck позволило обнаружить в моей программе стилистические ошибки и получить рекомендации по улучшению производительности. По результатам выполнения лабораторной работы я могу сказать, что CppCheck – удобный инструмент, с помощью которого можно легко и быстро обнаружить ошибки и недочеты в программе, которые лежат на поверхности.